Books to Life Pod

Project handoff for another ChatGPT instance | 10 October 2026 | Singapore

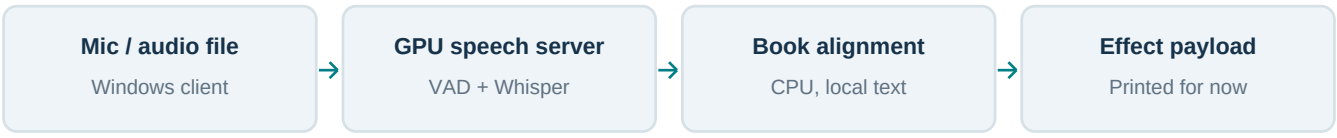
Current state

A working local Whisper prototype follows a supplied book passage, confirms scene checkpoints, and prints ambient effect payloads. Automatic moving book hints are implemented. Real child-reading accuracy, actual effect playback, and Jetson deployment still need validation.

What we are building

A capstone reading pod for a child aged about 5-10 and a parent/caregiver, in the context of work with NLB. Readers use a physical book. The app listens and follows their place in the story to trigger subtle lighting, sound, and visual ambience. Effects should be reusable across books and should support the reading experience.

- **Either reader counts.** A parent's correct reading can advance the story. Speaker identification is not required for effect triggers.
- **Skipping is acceptable.** Recover at a later scene without playing every missed effect in a burst. Checkpoint order and position matter.
- **Local, low latency runtime.** Cloud book preparation/catalogue is a future option; the active book is held in RAM. A local JSON cache can coexist with this.



Confirmed book progress also moves the prompt + hotwords sent to Whisper.

What has been completed

Area	Implemented / decided
Recognition	WhisperLive + faster-whisper; microphone and recorded-file input; VAD, transcript display, audio setup, and model preparation.
Progress	Full-passage word-sequence alignment, fuzzy word comparisons, confidence checks, wider recovery search, and at-most-once checkpoint handling.
Book context	Automatic nearby passage prompt plus current and two future checkpoint phrases as hotwords. Hints update with confirmed progress.
Deployment	Windows NVIDIA GPU server container with native client; separate Jetson and Pi profiles; README quickstart and two sample passages.

Repository snapshot

Repository: github.com/Kinosaurus/Speech-to-Text
Branch: **first-pipeline** | Latest commit: **c3e9e43** (automatic checkpoint hotwords)
Previous major commit: **b80ba96** (alignment and GPU profiles)
Workspace: `C:\Users\heart\Speech-to-Text`

At handoff, the source tree was clean and synchronized with origin. The intended GitHub account for future pushes is **how2fps**. No application changes are part of this PDF request.

Run and test on the PC

Two local programs: a Docker GPU server and a native Windows Python client

Current development hardware is an **RTX 4060 Laptop GPU** with approximately 8 GB dedicated VRAM. Docker Desktop must be running in Linux-container / WSL2 mode. Execute commands from the project folder. If the existing .venv already runs the client, skip installation.

One-time Windows client setup

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements-client.txt
.\.venv\Scripts\python.exe -m pip install whisper-live==0.10.0 --no-deps
```

Terminal 1: first-time GPU preparation, then start

```
docker compose -f compose.gpu.yaml build
docker compose -f compose.gpu.yaml run --rm asr python run_server.py `
  --device cuda --check_device
docker compose -f compose.gpu.yaml run --rm asr python prepare_models.py `
  --model small.en --device cuda --verify_inference
docker compose -f compose.gpu.yaml up
```

Run each preparation command to completion. On later sessions, use just the **up** command. The GPU server was rebuilt and restarted after adding live book hints. Following future source updates, use:

```
docker compose -f compose.gpu.yaml up -d --build asr
```

Terminal 2: choose live reading or a recording

```
# Microphone + the two Mermaid checkpoints
.\.venv\Scripts\python.exe run_client.py --model small.en --audio_chunk 1024 `
  --checkpoints .\examples\little_mermaid_story.json

# Recorded reading: replace this path with your own file
.\.venv\Scripts\python.exe run_client.py --model small.en `
  --files ".\recordings\LittleMermaid.wav" --mute_audio_playback `
  --checkpoints .\examples\little_mermaid_story.json
```

For **ASR-only tests**, omit `--checkpoints`. Add `--n_display_segments 20 --enable_timestamps` to see more transcript history. For a hints-off comparison, add `--no_book_context` to the same book-reading command.

Audio preparation and troubleshooting

- WAV, MP3, and M4A are accepted. Prefer **WAV, mono, 16000 Hz, signed 16-bit PCM** for repeatable tests. 44100/48000 Hz also works and is resampled. Higher export rates cannot restore lost recording detail.
- Audacity internal-audio recording: select **Windows WASAPI**, then the output device marked **(loopback)**; turn off input monitoring; record the playing video/audio; export audio. .aup4/.aup3 are project files, so export WAV first.
- `--mute_audio_playback` silences speakers only; the ASR still receives the file directly. Playing phone audio into a microphone adds acoustic distortion and is a different test.
- Port **9090** must be free. A Docker named-pipe error usually means the Desktop Linux engine is not running. Keep the model-cache volume; `down -v` deletes it. Ctrl+C stops foreground programs.

The default terminal view shows only four recent segments. Older text leaving that view is not evidence that the whole transcript changed. The session writes output.srt on exit; preserve it before another run overwrites it.

Book data and checkpoints

Full passage text is the alignment reference; checkpoints mark effect locations

The JSON now contains **pages[].text**: the words the readers are expected to read. This gives alignment context between checkpoints. It is stored locally as a manifest and loaded into an in-memory Python object during the session. Long books need not be sent wholesale to ASR; matching searches a local window and the Whisper prompt is bounded.

Field	Meaning
checkpoint_position	Scene/checkpoint order, such as 1 then 2. It is not a timestamp or word offset.
checkpoint_text + page	The anchor phrase and its logical page reference. Anchors should be distinctive.
word_start / word_end	Global normalized book-token offsets: zero-based, end exclusive. These are text positions, not audio timestamps.
effect	An independent payload describing ambience: effect ID, lighting, visuals, sound, fades, and per-effect delays in milliseconds.

Word ranges can be inferred when the anchor occurs uniquely in the stated page. Explicit ranges help disambiguate repeated wording. Existing Mermaid and Mila manifests include ranges. Model choice or target hardware does not require a different JSON schema.

Current reading test: The Little Mermaid

Files: **examples/little_mermaid_passage.txt** and **examples/little_mermaid_story.json**. Book ID: `little-mermaid-reading-demo`. The supplied excerpt has 56 normalized words and one logical page; this does not assert a printed page number.

Exact supplied passage

The next thing she knew, the Little Mermaid woke up. She had the prettiest legs on earth. And in front of her stood the handsome prince.

"Who are you" asked the Prince.
"Where do you come from?"
And the little mermaid could not speak, she could only look at the prince with her deep blue eyes.

Order / ID	Anchor and location	Planned ambience
1 mermaid_has_legs	"She had the prettiest legs on earth" Words [10, 17)	warm_daylight Warm colour, brightness 0.18, 1800 ms fade
2 mermaid_cannot_speak	"And the little mermaid could not speak" Words [37, 44)	quiet_blue_ambience Blue colour, brightness 0.14, 2000 ms fade

Visual files warm_light_loop.gif / soft_blue_loop.gif and sound light_breeze.wav are **placeholders**. The prototype prints their payloads; it does not play them. A growing-text simulation triggered both checkpoints in order with no skips. A real child's Mermaid recording has not yet established the same result.

Minimal checkpoint shape

```
{
  "id": "mermaid_has_legs", "checkpoint_position": 1, "page": 1,
  "checkpoint_text": "She had the prettiest legs on earth",
  "word_start": 10, "word_end": 17,
  "effect": {"effect_id": "warm_daylight", "fade_ms": 1800,
             "visual": "warm_light_loop.gif", "visual_delay": 0,
             "sound": "light_breeze.wav", "sound_delay": 0}
}
```

How tracking works today

Alignment improves context, but is still downstream of imperfect speech recognition

VAD identifies likely speech and filters silence; it does not identify book checkpoints. **ASR** converts audio to text. **Alignment** then estimates where that text belongs in the known passage. Continuous reading can span several checkpoints without a pause; detection depends on the transcript updates, not one silence per checkpoint.

Default word-sequence alignment

- Normalize punctuation/case and align recent transcript words in order against nearby book words, using local dynamic programming and fuzzy spelling similarity. Insertions, omissions, and imperfect words can still leave enough surrounding evidence.
- Search near the confirmed position; reject weak or ambiguous locations. A qualifying candidate is tentative until enough fresh updates agree. Confirmed progress moves forward, and each checkpoint is processed at most once per session.
- If the reader jumps ahead, confirm the later location, mark earlier unmatched checkpoints skipped, and emit the latest applicable effect. Following persistent misses, widen the forward search to recover beyond the first few checkpoints.

Default	Value / behavior
Matching mode	alignment; the older fuzzy checkpoint matcher remains selectable.
Search / transcript	120 book words ahead; 32 words back; recent 32 transcript words.
Evidence	At least 5 supported words and 3 distinct words; coverage >= 0.8; similarity score >= 82.
Confirmation	2 fresh qualifying updates for the same scene. Exact duplicate/trimmed messages do not add confirmations.
Recovery	After each 3 misses, expand forward range; recovery score >= 92; ambiguity margin 4. These are scores, not probability guarantees.

Reading terminal messages

Log	What it tells us
[TRANSCRIPT]	What ASR currently recognized; unfinished text can change as more audio arrives.
[ALIGN]	Estimated page/word span, confirmed progress, recovery, ambiguity, and duplicate status.
[MATCH]	Candidate score/confirmation state. This alone does not mean an effect fired.
[TRIGGER] / [SKIPPED]	An effect payload was emitted / an earlier checkpoint was passed without firing.

Count **[TRIGGER]** lines to count successful effects. Once every checkpoint is processed (triggered *or skipped*), transcription can continue while matching logs stop. Transcript-only output does not prove all effects fired.

Known weak points that are not fixed yet

- The matcher accepts provisional ASR text. A changed segment end timestamp or completed flag can count as a fresh update even when the relevant words have not changed. Confirmation is based on scene agreement, not fully stable aligned word positions.
- Unrelated comments can increase miss counts and widen recovery. Quoting a sufficiently long exact passage can trigger progress; transcript matching cannot reliably infer the speakers' intent.
- The four-word phrase 'this dragon is cool' falls below the default five-word minimum. Stutters and corrections can be tolerated through surrounding words, but there is no dedicated 'drag gon' to 'dragon' reconstruction or pronunciation scoring.

Both Whisper and alignment can fail. Larger models, soft hints, and stricter checks are tradeoffs to measure; none makes the system 100% reliable. The ambient use case permits recovery and missed scenes, but false scene jumps still need evaluation.

Latest work: book-aware Whisper

Completed, committed, and integrated into microphone and file paths

The ASR originally had no automatic knowledge of the selected book. It now receives **soft recognition hints**: a short surrounding passage as the initial prompt and distinctive upcoming checkpoint phrases as hotwords. These encourage expected wording without proving those words were actually spoken.

Automatic context behavior

- Enabled when `--checkpoints` is supplied, unless `--no_book_context` is set. Without a manifest, ASR-only tests receive no automatic book hints.
- Hotwords include the next expected checkpoint and up to two future checkpoints. Default prompt: 64 normalized words, with 8 words of lookback. Total prompt is capped at 480 characters; hotwords at 48 words / 320 characters; each phrase at 16 words / 100 characters.
- Refresh when the current checkpoint changes or confirmed word position advances another 16 words. Tentative matches do not move the hint window. The last checkpoint's hints remain available for the remaining passage.
- Startup hints are included in the connection handshake. A versioned live protocol sends later updates as text controls alongside binary PCM audio. The server queues an atomic prompt/hotword pair for the next inference; it does not reload model weights.
- The server advertises support before the client sends live controls. An older server gets startup hints only, with a client warning. Stale revisions cannot overwrite newer context. An acknowledgement means queued, not applied to inference already in progress.

Controls: `--book_context_lookahead` defaults to 2 (range 0-10); `--book_context_words` defaults to 64 (range 8-128). Explicit `--initial_prompt` or file overrides only the generated prompt; explicit `--hotwords` or file overrides only hotwords. Those overridden fields stay fixed.

Code map for the next assistant

Files	Responsibility
run_client.py / run_server.py	Command-line entry points; audio/session setup and the extended speech server.
alignment_matcher.py checkpoint_matcher.py	Full-book sequence tracking; manifest/base logic and legacy fuzzy fallback.
book_context.py	Bounded hint generation, refresh decisions, validation, per-session context state.
book_context_client.py book_context_server.py	Protocol negotiation and live updates; apply queued context on the ASR thread.
asr_runtime.py / audio_runtime.py prepare_models.py	Device/precision checks; input setup; model/VAD caching and inference check.
checkpoint_demo.py / tests/	Text-only demos and regression coverage. README.md has the current PC quickstart.

What has actually been verified

- The latest test suite contains **55 passing tests**. Actual GPU WebSocket testing negotiated live updates and acknowledged a moved Mermaid context; cached small.en CUDA inference accepted prompt and hotwords on silent input.
- PC GPU runtime reports CUDA / float16 on RTX 4060. Container stack: CUDA 12.3.2, Torch 2.5.1 with CUDA 12.1, CTranslate2 4.8.2. Native Windows .venv has CPU-only Torch; GPU work belongs in Docker.
- small.en and medium.en are cached. Synthetic inference checks of about 0.24 / 0.26 seconds established that inference runs; these are **not** child-reading accuracy or end-to-end effect latency measurements.

What we should do next

Recommended work, not a claim that it has already been implemented

1. Measure the current system before changing it

Start with the child's recording of the exact Mermaid excerpt and its two checkpoints. Run the same file with and without automatic context, keeping model, VAD, and matcher settings identical. Repeat with small.en / medium.en if memory and speed allow. Keep arbitrary ASR-only tests separate from book tracking.

Measure	How / why
Recognition	Manually write the actual spoken words, including omissions and corrections; compare raw ASR errors against that reference, separately from intended book text.
Effects	Label the expected scene crossings in the audio; record triggers, skips, misses, false triggers, and wrong scene jumps.
Response delay	Measure from spoken checkpoint evidence to emitted effect. Report median and p95 plus recording/device conditions; synthetic inference is insufficient.
Robustness	Try uninterrupted reading, repeated lines, stutters, parent correction, side talk, skipped pages, and repeated phrases at different locations.

2. Improve progress confirmation and recovery

- Keep tentative alignment separate from confirmed progress. Require compatible word positions and stable book-word evidence supported by genuinely new audio, not just a timestamp/completed-flag change. Allow a real finalized last sentence to complete without requiring unrelated new words.
- Do not interpret every off-book comment as proof of a skipped page. Require stronger distinctive context for large recovery jumps, while still allowing recovery when all three nearby checkpoints were missed.
- Handle restarts and corrections using book context without removing genuine repetition or dialogue. Exact quotations remain ambiguous; an optional pause/help control is a future product choice, not a solved inference problem.

3. Validate deployment and connect the actual effects

Target: **Jetson Orin Nano Super 8GB, JetPack 6.2 baseline**. Build its ARM64 GPU image, verify CUDA and the intended USB microphone/audio devices, then measure memory, heat, sustained speed, and offline model/book caching. Physical light/speaker/projector control and effect scheduling are still pending.

Platform	Current approach and limit
Windows PC	AMD64 Docker GPU server + native client. This path has been built and tested.
Jetson	Same app code; separate ARM64 / JetPack GPU image and audio client container. Actual board and GPU-image build are unverified.
Pi 4 / Pi 5	CPU int8 alternative. Their VideoCore GPU is not a CUDA backend for this ASR stack. Board performance is unverified.

PC 8 GB dedicated GPU VRAM and Jetson 8 GB shared system/GPU RAM are different resources. A working PC container cannot simply be assumed to run as the same binary image on Jetson. Compose profiles and setup files preserve the shared application logic.

4. Later catalogue and book preparation

Book identification, cloud catalogue retrieval, page OCR/model-assisted checkpoint selection, and approved content preparation are proposals. Keep full reference text plus deterministic scene positions; load only the selected book for runtime. NLB involvement does not establish that book-text storage permissions have already been resolved.

Alternatives and continuation brief

Preserve the user's decisions and keep evidence separate from proposals

ASR alternatives discussed, not integrated

Candidate	Why consider it / integration implication
Whisper / faster-whisper	Current baseline with a working streaming wrapper and new book hints. Existing prototype familiarity drove its use; it was not proven best for children.
Parakeet Unified English 0.6B	First suggested comparison candidate. Uses NVIDIA NeMo and its own streaming/decoder interface. Needs separate dependencies and an adapter; changing --model alone is insufficient.
Qwen3-ASR	Multilingual alternative with a different runtime. Its official streaming path uses vLLM; evaluate hardware fit and streaming behavior before integrating.
Cohere Transcribe / VibeVoice	Possible offline reference comparisons, with larger runtime/memory costs. Published general benchmarks do not establish child-reading results.
Moonshine	Historical lightweight/streaming candidate in project notes; not benchmarked in this project.

NeMo supports phrase/word boosting for suitable decoder paths, a counterpart to book-hint biasing. The exact model/version, streaming method, and Jetson combination still need validation. Proposed approach: isolate a file-based comparison first, then integrate a winner through a backend adapter while preserving the existing manifest and matcher.

Other discussions to preserve

- The university offered shared education GPU resources. A provisional request was 8 CPU cores, 24 GB RAM, one shared 20-24 GB VRAM GPU instance, and 100 GB home storage; a 16 GB instance could start smaller-model experiments. This was a starting allocation, not a measured minimum or a claim that access is granted.
- Pronunciation grading would require a separate audio/phoneme assessment pipeline. The current transcript matcher does not grade pronunciation, distinguish a quote from reading, or identify a child versus parent. Either reader counts for the ambient effects.
- A drag-and-drop browser testing UI was requested and then explicitly cancelled ('nvm scrap it'). Do not resume it without a new request. No Parakeet integration, cloud service, or physical effect playback has been completed.

Instructions for the receiving ChatGPT instance

Suggested continuation

Use this document as context, then inspect the current repository before making changes. Preserve shared reading, subtle reusable ambience, at-most-once effects, and later-scene recovery. Start with real-recording evaluation of the implemented Whisper hints. Treat progress-guard changes and alternative ASR integration as proposals requiring the user's chosen next task. Do not promise 100% accuracy or equate synthetic GPU timing with pod response latency.

Useful local references: README.md, JETSON_SETUP.md, PI4_SETUP.md, BOOKS_TO_LIFE_POD_NOTES.md, app_flow.drawio, simple_app_flow.drawio, and checkpoint_flow.mmd. The alternative Mila passage is examples/sample_passage.txt + examples/sample_story.json (four checkpoints). Older notes contain historical Pi/Moonshine estimates; current code and measured results supersede stale status claims. Existing dist ZIP bundles may predate the context updates.

Primary references for follow-up research

faster-whisper implementation and usage

Docker Desktop GPU support on Windows

Audacity desktop audio recording

Parakeet Unified English model card

NeMo word/phrase boosting documentation

Qwen3-ASR official repository